

Тестовое задание: QA Engineer (Middle+/Senior)

Рад, что ты откликнулся на нашу вакансию!

Мы ищем инженера, который не просто знает классическую теорию и практику тестирования, но и умеет использовать AI-инструменты (ChatGPT, Claude, Cursor, Copilot и др.) как «вторые руки» для кратного ускорения рутины.

Это комплексное тестовое задание рассчитано на **2–3 дня** самостоятельной работы. Его главная цель — показать твой инженерный кругозор, понимание интеграций, включая 1С и твою культуру работы с AI.

Формат сдачи задания

Мы оцениваем результаты так же, как оценивали бы реальные задачи в рамках рабочего спринта.

Никаких Google Документов или PDF-файлов — только чистый инженерный подход:

1. **Используй свой репозиторий** на GitHub или GitLab.
2. **Раздели артефакты:**
 - Все промпты, вывод ИИ и твои ответы на архитектурные кейсы должны быть оформлены в виде отдельных Markdown (.md) файлов в этом же репозитории
 - Под каждую задачу создай отдельную директорию с [readme.md](#) файлом, где описан ход решения
3. **Пришли нам ссылку** на репозиторий или на Pull/Merge Request, если хочешь показать историю коммитов для ревью.
Если в процессе выполнения возникнут вопросы по бизнес-логике или техническим ограничениям — создавай Issue в своем репозитории, в процессе review ответов мы посмотрим их.

Условия выполнения: обязательная часть и блоки на выбор

- **Обязательная часть:** блоки 1 (ЭТрН) и блок 2 (AI Prompting)
- **Блоки на выбор,** выбери любые два:
 - 3: Анализ данных (SQL)
 - 4. Тестирование API
 - 5. Базовые вопросы про 1С и структуру обменов

1. Блок: Тестирование требований (Бизнес-процесс ЭТрН)

Задача: Вам предлагается выполнить практическое тестовое задание по функциональности электронной транспортной накладной (ЭТрН), которая законодательное вступает в силу с 01 сентября 2026.

На основе краткого описания бизнес-процесса, которое прилагается к заданию, необходимо:

- Составить чек-лист или набор тест-кейсов для проверки основного бизнес-процесса ЭТрН: создание, валидации полей, смена статусов, отображение реквизитов, обработка ошибок (*подсказка: смотри BPMN процесс*)
- Описать ключевые проверки для формы/документа (*подсказка: смотри xsd-схему*)
- Привести несколько негативных проверок с указанием ожидаемого поведения системы и оформите один-два баг-репорта в произвольном, но структурированном виде (заголовок, шаги воспроизведения, фактический и ожидаемый результат, серьезность).

Объем и оформление — на ваше усмотрение, главное — содержательность и логика.

Контекст:

- BPMN-диаграмма процесса работы с электронной транспортной накладной: https://disk.360.yandex.ru/i/ob6qR1zF2NN_qA
- Форматы электронных договоров перевозки грузов: описание и xsd схема https://www.nalog.gov.ru/rn77/about_fts/docs/14975031/

Критерии оценки

- **Полнота покрытия** - охвачены ли основные сценарии: happy path, граничные значения
- **Понимание бизнес-процесса** - прослеживается ли логика ЭТрН, а не формальный перебор полей
- **Качество баг-репортов** - точность формулировок, корректность severity/priority

2. Блок: Проектирование тестов (AI Prompting & Review)

Мы хотим проверить, как ты генерируешь фильтры для тестирования с помощью ИИ и как фильтруешь «мусор».

1. **Задача:** С помощью любой LLM сгенерируй чек-лист для тестирования формы описанной ниже (позитивные, негативные сценарии, граничные значения).
2. **Контекст:** мы разрабатываем MVP формы создания заявки на перевозку для «УЛ Лайт». В форме есть следующие поля:
 - **Город отправления** (автокомплит/выпадающий список из базы данных).
 - **Город назначения** (автокомплит/выпадающий список из базы данных).
 - **Вес груза** (числовое поле, в килограммах, максимум 20 000 kg).
 - **Объем груза** (числовое поле, в кубических метрах, максимум 82 м³).
 - **Дата погрузки** (датапикер, нельзя выбрать прошедшую дату).

- Кнопка **«Рассчитать стоимость»** (отправляет POST-запрос к API, возвращает цену в рублях или ошибку, если маршрут не найден).
3. **Что опубликовать в репозитории:**
- **Твой промпт:** Покажи, как именно ты составил контекст и задачу для нейросети.
 - **Сырой ответ ИИ:** Скопированный без изменений текст, который выдала модель.
 - **Твое ревью ответа:** Проанализируй результат.
Укажи, где ИИ ошибся, что забыл учесть (специфику логистики, UX, особенности интеграции) и какие проверки ты бы удалил как избыточные или оторванные от реальности. **Это ключевая часть блока.**
 - **Файл с граничными условиями для тестирования** и краткое пояснение, почему именно эти кейсы критичны.

3. Блок: Анализ данных (SQL)

Напиши SQL-запросы для следующих задач (диалект PostgreSQL):

- **Задача: Вторая зарплата.**
Дана таблица Employee (id int PK, salary int).
Напиши SQL-запрос, который находит **вторую по величине уникальную** зарплату.
Если её нет, запрос должен вернуть null.
- **Задача: Поиск дубликатов email.**
Дана таблица Person (id int PK, email varchar).
Напиши SQL-запрос, который находит все **дублирующиеся** email-адреса.
- **Задача: Клиенты без заказов.**
Дана таблица Customers (id int PK, name varchar) и Orders (id int PK, customerId int FK). Напиши SQL-запрос, который возвращает всех клиентов, которые **ни разу не делали заказов**.

Рекомендация / подсказка: потренироваться в решении задач на *leetcode*
<https://leetcode.com/problemset/database/>

4. Блок: Тестирование API

Мы проектируем наши API-сервисы, опираясь на индустриальные стандарты: *Google REST API Guide*, *Microsoft REST API Guidelines*

1. **Контекст:** у тебя есть контроллер `/api/v2/orders`, управляющий жизненным циклом заказа и написанный в соответствии со стандартами индустрии.
Ссылка на md файл с описанием: <https://disk.360.yandex.ru/i/50qskYKTwh3b6A>

Задача: составить чек-лист, проверяющий корректное поведение контроллера.
Явно укажи, какие статус-коды ожидаются при успешном выполнении, ошибках

валидации, отсутствии прав доступа, работе с несуществующим ID и падении БД.

Рекомендация / подсказка: посмотреть оригинальный гайдаины, в них хорошо описаны сценарии использования, *http verbs* и *http status codes*

- <https://cloud.google.com/apis/design>
- <https://github.com/microsoft/api-guidelines/blob/vNext/graph/GuidelinesGraph.md>

5.Блок: Базовые вопросы про 1С

«Умная Логистика» работает на платформе 1С и нам важно, чтобы ты умел работать в продуктах экосистемы 1С.

Контекст: дана таблица (виртуальная) – регистр сведений «Цены доставки».

Период	Тип клиента
01.01.2025	Юр
01.01.2025	Юр
01.01.2025	Физ
01.02.2025	Юр

Задача: Напишите **запрос на языке запросов 1С** (или псевдокод с пояснением логики), который для заданных параметров (Дата заказа, Тип клиента, Общий вес) вернёт актуальную стоимость доставки. Учитывайте:

- выбор записи с максимальной датой начала, меньшей или равной дате заказа;
- корректную обработку границ веса (вес = 10 – должен брать интервал до 10, а не от 10; обратите внимание на отсутствие перекрытия);
- случай, если вес выходит за тах известный интервал – тогда стоимость не определена (вернуть NULL/0).

Контекст : Вам дана вымышленная упрощённая реализация метода **РассчитатьСтоимость**:

<

Функция РассчитатьСтоимость()

СуммаВес = 0;

Для каждого Строка из ТабличнаяЧасть.Номенклатура Цикл

СуммаВес = СуммаВес + Строка.Вес;

КонецЦикла;

```

Запрос = Новый Запрос;
Запрос.Текст = "ВЫБРАТЬ
|  ЦеныДоставки.Стоимость КАК Стоимость
|ИЗ
|  РегистрСведений.ЦеныДоставки КАК ЦеныДоставки
|ГДЕ
|  ЦеныДоставки.ТипКлиента = &ТипКлиента
|  И &СуммаВес МЕЖДУ ЦеныДоставки.ВесОт И ЦеныДоставки.ВесДо
|  И ЦеныДоставки.Период <= &ДатаЗаказа
|УПОРЯДОЧИТЬ ПО
|  Период УБЫВ";

Запрос.УстановитьПараметр("ТипКлиента",
СправочникКлиенты.НайтиПоРеквизиту(...));
Запрос.УстановитьПараметр("СуммаВес", СуммаВес);
Запрос.УстановитьПараметр("ДатаЗаказа", ТекущаяДата());

Результат = Запрос.Выполнить();
Если Результат.Пустой() Тогда
    Возврат 0;
Иначе
    Выборка = Результат.Выбрать();
    Выборка.Следующий();
    Возврат Выборка.Стоимость;
КонецЕсли;
КонецФункции
>

```

Задача: Найдите в этом коде не менее 5 ошибок/потенциальных проблем.

Опционально: Предложите исправления этих ошибок

Чек-лист самопроверки перед сдачей

- [] Репозиторий публичный и доступен для чтения.
- [] Файлы оформлены валидным Markdown, текст легко читается, SQL-запросы обернуты в блоки кода.
- [] Везде, где применялся ИИ, видна твоя собственная экспертная оценка, а не слепое копирование.

Если в процессе выполнения возникнут вопросы по бизнес-логике или техническим ограничениям — создавай Issue в своем репозитории, в процессе review ответов мы посмотрим их.

Желаем удачи!